COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S4
DATE PERFORMED:	7/24/2026	**DATE SUBMITTED:**	7/29/2026
NAME:	Jerry Matthew L. Larracas	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. _3.1_ LINKED LISTS

1. PROCEDURE OUTPUT

Screenshot

```
#ifndef SINGLY_LL_H
#define SINGLY_LL_H

template <typename T>
class SinglyList{
    public:
    T data;
    SinglyList<T>* next;
};
```

```
#include <iostream>
#include "singly_ll.h"


int main(){
    SinglyList<char>* head = nullptr;
    SinglyList<char>* second = nullptr;
    SinglyList<char>* third = nullptr;
    SinglyList<char>* fourth = nullptr;
    SinglyList<char>* fifth = nullptr;


    //allocated memory to the head
    head = new SinglyList<char>;
    second = new SinglyList<char>;
    third = new SinglyList<char>;
    fourth = new SinglyList<char>;
    fifth = new SinglyList<char>;

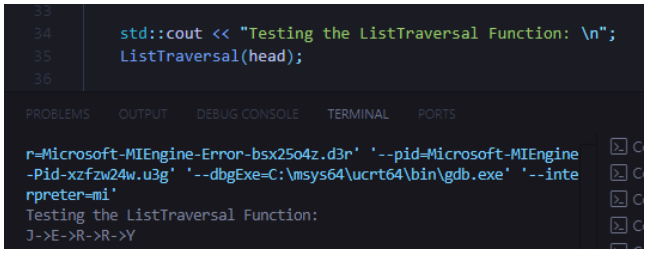
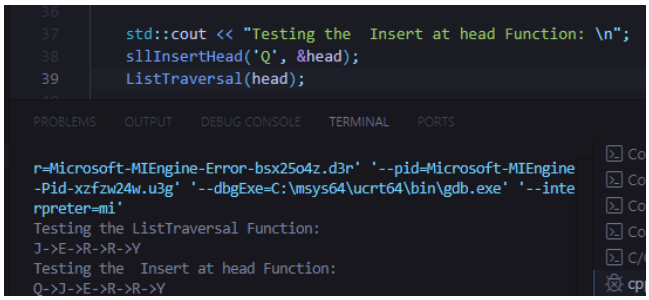
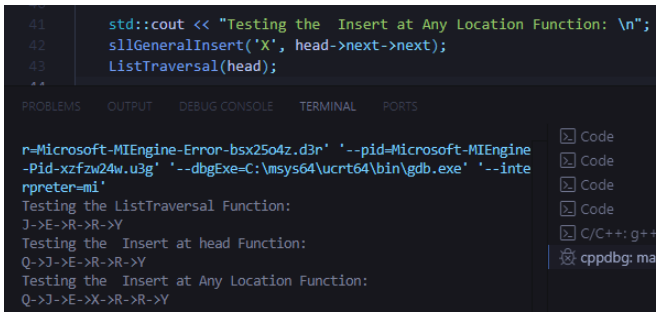
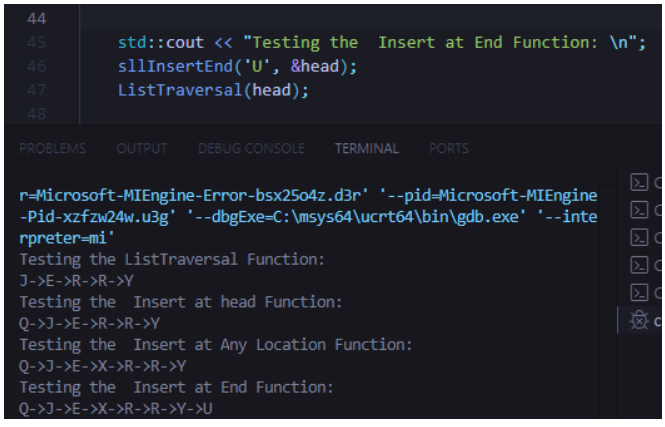
    //added the data to the node
    head->data = 'J';
    second->data = 'E';
    third->data = 'R';
    fourth->data = 'R';
    fifth->data = 'Y';

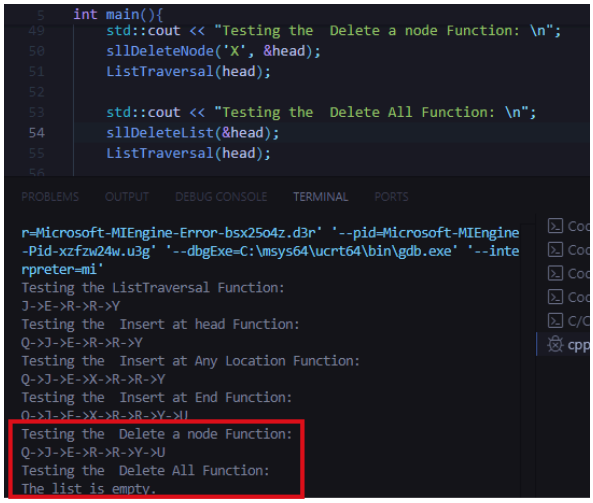
    //assign the next to the node
    head->next = second;
    second->next = third;
    third->next = fourth;
    fourth->next = fifth;
    fifth->next = nullptr;
```

Discussion

After we run the code, it didn't show or display anything

Table 3-1. Output of Initial/Simple Implementation

Operation	Screenshot
Traversal	
Insertion at head	
Insertion at Any part of the list	
Insertion at End	

Deletion of a node	

```
int main(){
    std::cout << "Testing the  Delete a node Function: \n";
    sllDeleteNode('X', &head);
    ListTraversal(head);

    std::cout << "Testing the  Delete All Function: \n";
    sllDeleteList(&head);
    ListTraversal(head);
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

r=Microsoft-MIEngine-Error-bsx25o4z.d3r' '--pid=Microsoft-MIEngine
-Pid-xzfzw24w.u3g' '--dbgExe=C:\msys64\ucrt64\bin\gdb.exe' '--inte
rpreter=mi'
Testing the ListTraversal Function:
J->E->R->R->Y
Testing the  Insert at head Function:
Q->J->E->R->R->Y
Testing the  Insert at Any Location Function:
Q->J->E->X->R->R->Y
Testing the  Insert at End Function:
Q->J->E->X->R->R->Y->U
Testing the  Delete a node Function:
Q->J->E->R->R->Y->U
Testing the  Delete All Function:
The list is empty.
```

Table 3-2. Code for the List Operations

a.	**Source Code**	

```
template <typename T>
void ListTraversal(SinglyList<T>* node){

    // if the list is empty:
    if(node == nullptr){
        std::cout << "The list is empty. \n";
        return;
    }

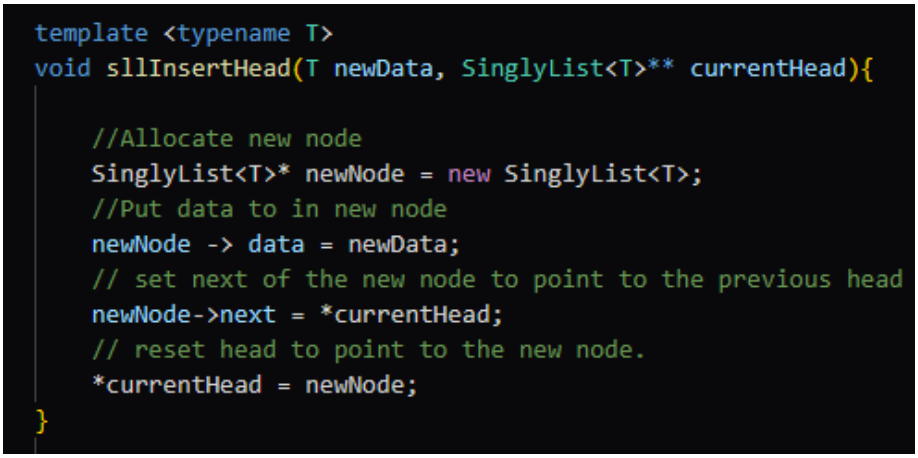
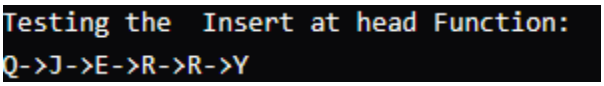
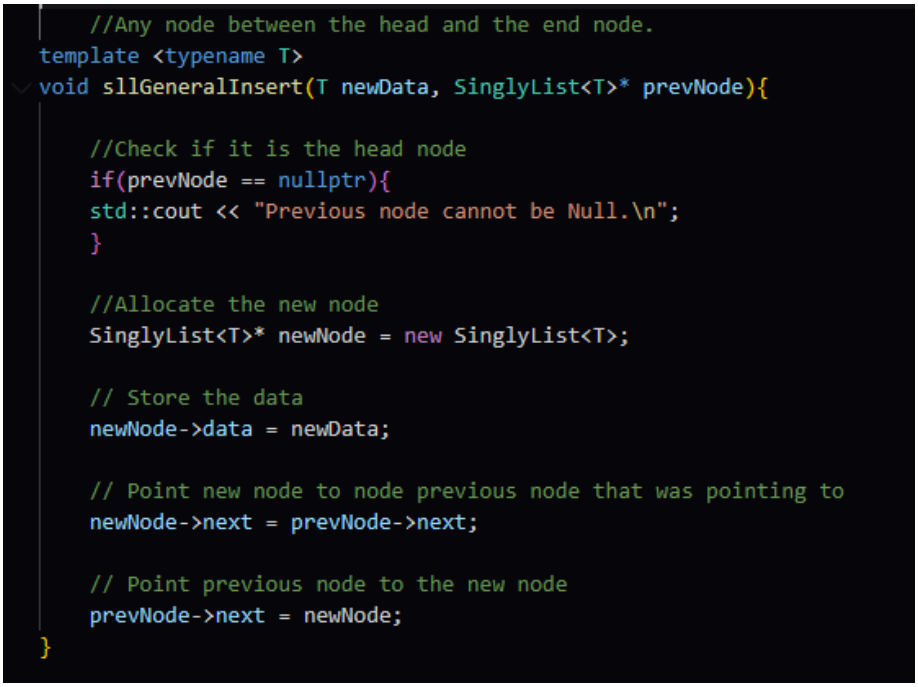
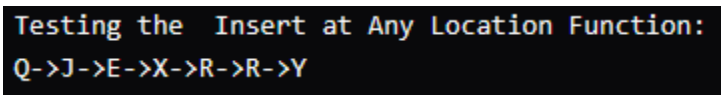
    while(node != nullptr){
        std::cout << node->data;

        if(node->next != nullptr){
            std::cout << "->";
        }

        node = node->next;
    }
    std::cout << std::endl;
}
```

	Console	

```
Testing the ListTraversal Function:
J->E->R->R->Y
```

b.	**Source Code**	
	Console	
C.	**Source Code**	
	Console	

```
template <typename T>
void sllInsertHead(T newData, SinglyList<T>** currentHead){

    //Allocate new node
    SinglyList<T>* newNode = new SinglyList<T>;
    //Put data to in new node
    newNode -> data = newData;
    // set next of the new node to point to the previous head
    newNode->next = *currentHead;
    // reset head to point to the new node.
    *currentHead = newNode;
}
```

```
Testing the  Insert at head Function:
Q->J->E->R->R->Y
```

```
    //Any node between the head and the end node.
template <typename T>
void sllGeneralInsert(T newData, SinglyList<T>* prevNode){

    //Check if it is the head node
    if(prevNode == nullptr){
    std::cout << "Previous node cannot be Null.\n";
    }

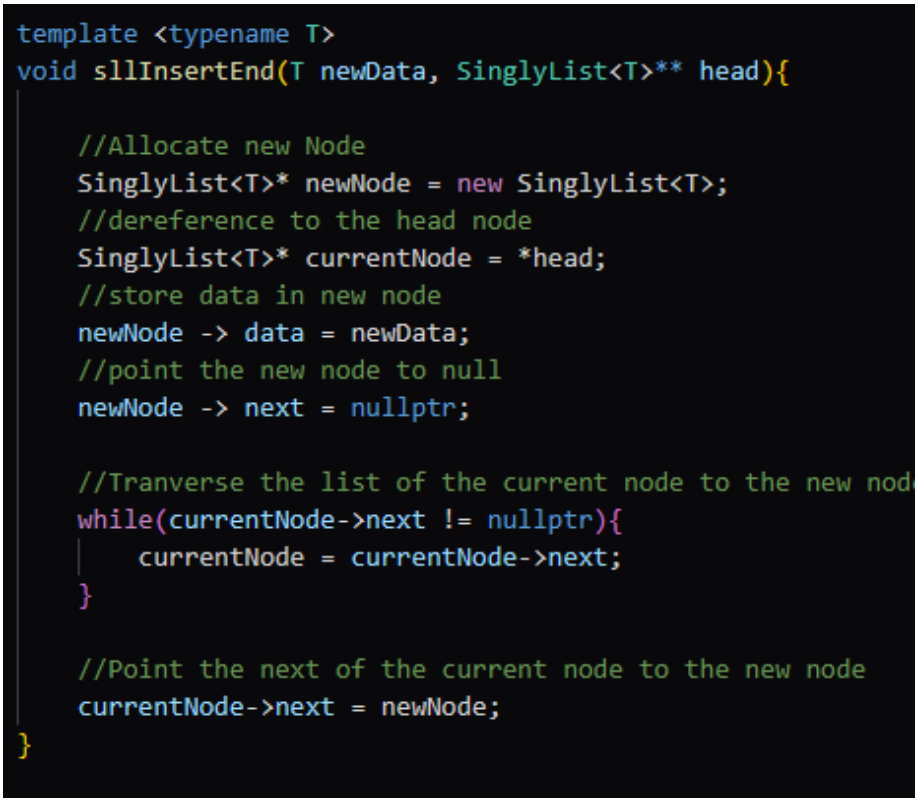
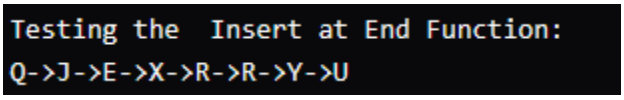
    //Allocate the new node
    SinglyList<T>* newNode = new SinglyList<T>;

    // Store the data
    newNode->data = newData;

    // Point new node to node previous node that was pointing to
    newNode->next = prevNode->next;

    // Point previous node to the new node
    prevNode->next = newNode;
}
```

```
Testing the  Insert at Any Location Function:
Q->J->E->X->R->R->Y
```

d.	**Source Code**	
	Console	

```
template <typename T>
void sllInsertEnd(T newData, SinglyList<T>** head){

    //Allocate new Node
    SinglyList<T>* newNode = new SinglyList<T>;
    //dereference to the head node
    SinglyList<T>* currentNode = *head;
    //store data in new node
    newNode -> data = newData;
    //point the new node to null
    newNode -> next = nullptr;

    //Tranverse the list of the current node to the new nod
    while(currentNode->next != nullptr){
        currentNode = currentNode->next;
    }

    //Point the next of the current node to the new node
    currentNode->next = newNode;
}
```

```
Testing the  Insert at End Function:
Q->J->E->X->R->R->Y->U
```

<table>
<tr><td rowspan="2">e.</td><td>Source Code</td><td>

```
template <typename T>
void sllDeleteNode(T findData, SinglyList<T>**  head){

    if(*head == nullptr) return;

    SinglyList<T>* currNode = *head;
    SinglyList<T>* prevNode = nullptr;

    // Search for the data
    while(currNode != nullptr && currNode->data != findData){
        prevNode = currNode;
        currNode = currNode->next;
    }
    // Data not found
    if(currNode == nullptr) return;

    if(prevNode == nullptr){
        *head = currNode->next;
    } else{
        prevNode->next = currNode->next;
    }

    delete currNode;
}
```

</td></tr>
<tr><td>Console</td><td>

```
Testing the  Delete a node Function:
Q->J->E->R->R->Y->U
```

</td></tr>
<tr><td rowspan="2">f.</td><td>Source Code</td><td>

```
    template <typename T>
    void sllDeleteList(SinglyList<T>** head){
    SinglyList<T>* current = *head;

    while(current != nullptr){
        SinglyList<T>* temp = current;
        current = current->next;
        delete temp;
    }

    *head =  nullptr;

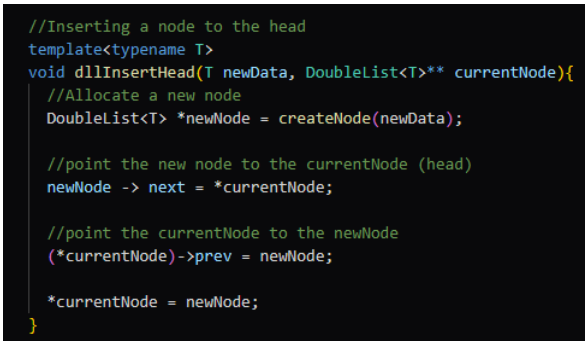
}
```

</td></tr>
<tr><td>Console</td><td>

```
Testing the  Delete All Function:
The list is empty.
```

</td></tr>
</table>

Table 3-3. Code and Analysis for Singly Linked Lists

Screenshots	Analysis
	This function checks the list if it is empty and also lists the forwarded and reversed of a list. The tail is the one who is reversing the data and the forwarding is the one who is doing the forward of the data
	This function purpose is to insert a data at the head or at the first node but creating a new node and inserting it at the first node.

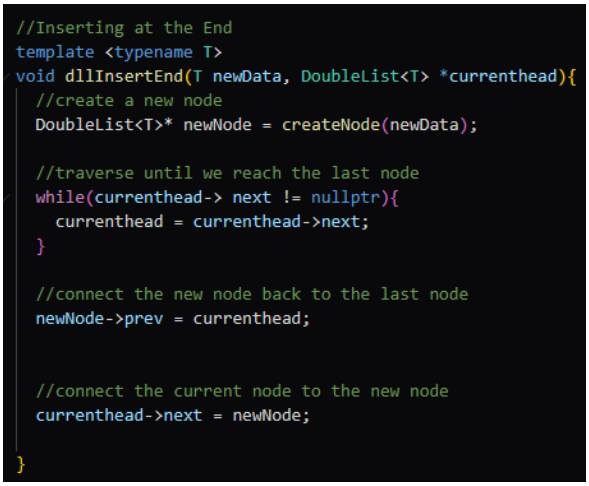
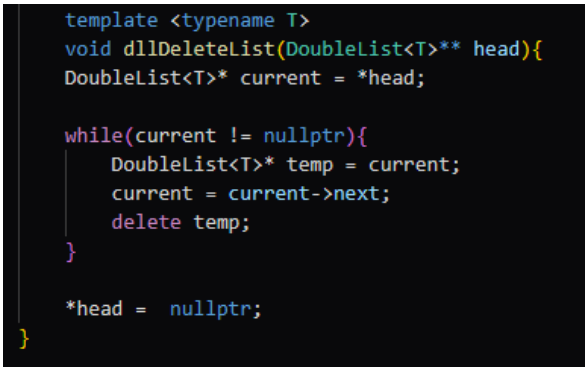
	This is the opposite of the inserting at the head, this time this function inserts at the end, or at the last node. It also creates a new node and insert at the last node the new data or the new node.
	This function deletes the existing nodes.
	The purpose of this function is to shows if the list is truly empty.

Table 3-4. Modified Operations for Doubly Linked Lists

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Problem Title: Implementing a Song Playlist using Linked List
Source: Packt Publishing
Problem Description:
In this activity, we'll look at some applications for which a singly linked list is not enough or not convenient. We will build a tweaked version that fits the application. We often encounter cases where we have to customize default implementations, such as when looping songs in a music player or in games where multiple players take a turn one by one in a circle.
These applications have one common property – we traverse the elements of the sequence in a circular fashion. Thus, the node after the last node will be the first node while traversing the list. This is called a circular linked list.
We'll take the use case of a music player. It should have following functions supported:

- Create a playlist using multiple songs.
- Add songs to the playlist.
- Remove a song from the playlist.
- Play songs in a loop (for this activity, we will print all the songs once).
- Here are the steps to solve the problem:
- Design the basic structure that supports circular data representation.
- After that, implement the insert and delete functions in the structure.
- Implement a function for traversing the playlist.

The driver function should allow for common operations on a playlist such as: next, previous, play all songs, insert and remove.

Source Code

```
#include <iostream>
#include <string>
#include "doubly_ll.h"


int main(){

    DoubleList<std::string>* head = createNode(
        std::string("Rob Daniel - Nandito ako"));
    DoubleList<std::string>* second = createNode(
        std::string("Hale - Kung wala ka"));
    DoubleList<std::string>* third = createNode(
        std::string("Soapdish - Tensionado"));

    head->next = second;
    second->next = third;

    second->prev = head;
    third->prev = second;

    int choice;
    std::string song;

do {

  std::cout << "\n ===== Jerry's Song Playlist ===== \n";
    dllTraversal(head);

    std::cout<<"\n\n==== OPTIONS ====\n";
    std::cout<< "[1] Play song playlist\n";
    std::cout<< "[2] Add songs to the playlist\n";
    std::cout<< "[3] Remove a song from the playlist\n";
    std::cout<< "[4] Exit\n\n";
    std::cout<< "Enter choice: ";
    std::cin >> choice;
    std::cin.ignore();

    switch (choice){
    case 1:
        std:: cout << "\nNow playing... \n";
        dllTraversal(head);
        break;

    case 2:
```

```
    case 2:
        std::cout<< "Enter a song: ";
        getline(std::cin, song);
        dllInsertEnd(std::string(song), head);
        std::cout << "\nSong added successfully\n";
        break;

    case 3:
        std::cout << "Select a song you want to remove: \n";
        std::cin >> song;
        break;

    case 4:
        std::cout << "Shutdowning Music Player...\n";
        break;

    default:
        std::cout<<"invalid choice!\n";
    }
   } while(choice != 4);

   return 0;
}
```

```
#ifndef DOUBLY_LL_H
#define DOUBLY_LL_H

//Doublelist Node
template <typename T>
class DoubleList{
  public:
  T data;
  DoubleList<T>* next;
  DoubleList<T>* prev;
};

//creating a node function
template <typename T>
DoubleList<T>* createNode(T newData){

    //dynamic memory allocation for a new node
    DoubleList<T>* newNode = new DoubleList<T>;

    newNode->data = newData;
    //point it to the next node
    newNode->prev = nullptr;
    newNode->next = nullptr;

    return newNode;
}

template<typename T>
void dllTraversal(DoubleList<T>* currentNode){
  // check if the list is empty
  if(currentNode == nullptr){
    std::cout << "The list is empty";
    return;
  }

  DoubleList<T>* tail;
  std::cout << "";
  //forward traverse
  while(currentNode != nullptr){
    std::cout << currentNode->data << "\n";
    tail = currentNode;
    currentNode = currentNode -> next;
  }
  //reverse traversal
```

```cpp
//Inserting a node to the head
template<typename T>
void dllInsertHead(T newData, DoubleList<T>** currentNode){
  //Allocate a new node
  DoubleList<T> *newNode = createNode(newData);

  //point the new node to the currentNode (head)
  newNode -> next = *currentNode;

  //point the currentNode to the newNode
  (*currentNode)->prev = newNode;

  *currentNode = newNode;
}

//Inserting at the End
template <typename T>
void dllInsertEnd(T newData, DoubleList<T> *currenthead){
  //create a new node
  DoubleList<T>* newNode = createNode(newData);

  //traverse until we reach the last node
  while(currenthead-> next != nullptr){
    currenthead = currenthead->next;
  }

  //connect the new node back to the last node
  newNode->prev = currenthead;


  //connect the current node to the new node
  currenthead->next = newNode;

}


#endif
```

Console

```
 ===== Jerry's Song Playlist =====
Rob Daniel - Nandito ako
Hale - Kung wala ka
Soapdish - Tensionado


==== OPTIONS ====
[1] Play song playlist
[2] Add songs to the playlist
[3] Remove a song from the playlist
[4] Exit

Enter choice: 1

Now playing...
Rob Daniel - Nandito ako
Hale - Kung wala ka
Soapdish - Tensionado

 ===== Jerry's Song Playlist =====
Rob Daniel - Nandito ako
Hale - Kung wala ka
Soapdish - Tensionado
```

```
 ===== Jerry's Song Playlist =====
Rob Daniel - Nandito ako
Hale - Kung wala ka
Soapdish - Tensionado


==== OPTIONS ====
[1] Play song playlist
[2] Add songs to the playlist
[3] Remove a song from the playlist
[4] Exit

Enter choice: 2
Enter a song: Passionfruit

Song added successfully

 ===== Jerry's Song Playlist =====
Rob Daniel - Nandito ako
Hale - Kung wala ka
Soapdish - Tensionado
Passionfruit


==== OPTIONS ====
[1] Play song playlist
[2] Add songs to the playlist
[3] Remove a song from the playlist
[4] Exit
```

Explanation: In this supplementary activity I managed to do it but I wasn't able to implement remove a song from the playlist or next or prev feature. But as we can see I managed to add the option of playing the song playlist and add songs to the playlist. I added a switch case function in order to select whenever you want to play the song playlist, add songs and remove songs, and also exit the system.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.